

Robô Seguidor de Linha

Documentação técnica: hardware, firmware e uso

Sumário

1. Visão geral	2
1.1 Resumo dos números principais	2
2. Arquitetura do sistema	2
3. Componentes de hardware	3
3.1 Microcontrolador – STM32F401CCU6 (“Black Pill”)	3
3.2 Barra de sensores	3
3.3 Driver de motores – DRV8833	3
3.4 Motores	4
3.5 ESP32-C3	4
3.6 Alimentação	4
3.7 Pinagem completa	4
4. Firmware do STM32	4
4.1 Organização dos arquivos	4
4.2 Temporização e laço principal	5
4.3 Aquisição dos sensores	5
4.4 Normalização, posição e linha perdida	5
4.5 Controlador PID	6
4.6 Dos números ao motor: mistura, rampa e zona morta	6
4.7 Máquina de estados	7
4.8 Protocolo serial	7
4.9 Calibração dos sensores	8
5. Firmware do ESP32-C3 (ESP32C3_Tuner)	8
6. Parâmetros ajustáveis	9
7. Procedimentos de uso	9
7.1 Primeira ligação	9
7.2 Sensores	9
7.3 Sintonia do PID	9
8. Referências	10

1. Visão geral

O projeto é um robô seguidor de linha de duas rodas com tração diferencial. Um microcontrolador **STM32F401CCU6** lê uma barra de **8 sensores refletivos analógicos**, calcula a posição da linha, roda um controlador **PID** a 500 Hz e aciona dois motores DC por um driver **DRV8833**. Um **ESP32-C3** separado funciona como ponte de ajuste: cria uma rede Wi-Fi com uma página web onde é possível mudar os ganhos do PID, a velocidade, testar motores, ler e calibrar sensores, tudo sem regravar o firmware.

O repositório tem duas partes:

Pasta	Ferramenta	Função
LineFollowerRobot/	STM32CubeIDE (C, HAL da ST, gerado pelo CubeMX)	Firmware do robô: sensores, controle, motores, protocolo serial.
ESP32C3_Tuner/	Arduino IDE (core ESP32)	Ponte Wi-Fi ↔ UART com página de calibração.
PINOUT.md / .csv	–	Mapa de pinos e resumo do protocolo.

1.1 Resumo dos números principais

Item	Valor	Origem
Clock do sistema	84 MHz (HSI 16 MHz + PLL)	main.c: SystemClock_Config
Período de controle	2 ms (500 Hz)	TIM4: 84 MHz / 84 / 2000
PWM dos motores	20 kHz, 4200 passos	TIM2/TIM3: ARR = 4199, PSC = 0
ADC	12 bits, 8 canais, ~59 µs por varredura	21 MHz, 144 + 12 ciclos por canal
Faixa de posição	0..7000, centro 3500	POSITION_CENTER
Escala de comando (U)	-1000..+1000 (= -100%..+100%)	U_MAX
Velocidade base padrão	595 U (59,5%)	VEL_BASE_PADRAO
PID padrão	Kp 0,1 · Ki 0 · Kd 0,05	main.c:177
Link serial	USART1, 115200 8N1, erro de baud ≈ 0,02%	comm.c

2. Arquitetura do sistema

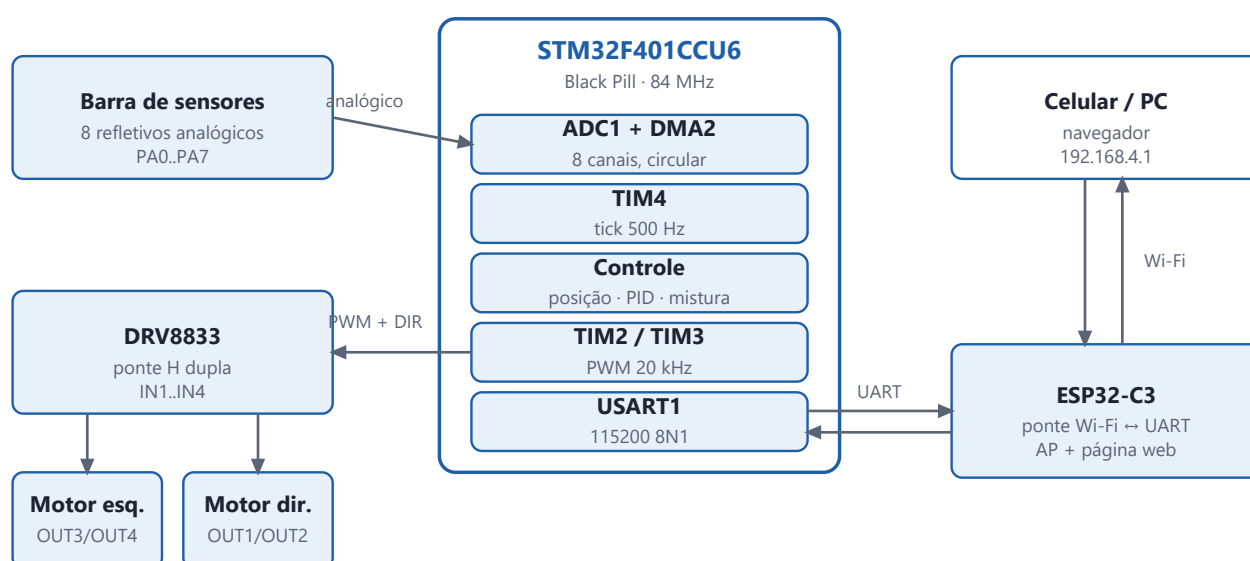


Figura 1 – Blocos do sistema e ligações. O ESP32-C3 não participa do controle; ele só envia comandos e mostra as respostas.

Toda a malha de controle roda no STM32. O ESP32-C3 pode ser removido na competição: o robô continua funcionando com os valores que estão na RAM ou nos padrões do firmware. Se o ESP estiver ligado, ele

também pode parar o robô quando a página web perde a conexão (ver seção 5).

3. Componentes de hardware

3.1 Microcontrolador – STM32F401CCU6 (“Black Pill”)

Cortex-M4F com FPU de precisão simples (por isso o PID usa `float` sem custo relevante), 256 KB de Flash e 64 KB de RAM, encapsulamento UFQFPN48. O clock vem do oscilador interno HSI (16 MHz) multiplicado pelo PLL ($M = 8$, $N = 84$, $P = 2$) para 84 MHz; APB1 a 42 MHz e APB2 a 84 MHz. O regulador está em *voltage scale 2*, adequado para 84 MHz.

Periférico	Configuração	Uso
ADC1	12 bits, scan + contínuo, PCLK2/4 = 21 MHz, 144 ciclos de amostragem	Lê os 8 sensores (PA0..PA7)
DMA2 Stream0	Circular, half-word, interrupções desligadas	Copia o ADC para <code>line_sensor_raw[8]</code> sem usar a CPU
TIM2 CH2	PWM modo 1, 20 kHz	Motor esquerdo (PB3)
TIM3 CH2	PWM modo 1, 20 kHz	Motor direito (PB5)
TIM4	PSC 83, ARR 1999 → interrupção a 500 Hz	Base de tempo do controle
USART1	115200 8N1, configurado por registrador em <code>comm.c</code>	Link com o ESP32-C3
GPIO	PB4, PB6 saída push-pull	Direção/freio dos motores

3.2 Barra de sensores

Oito sensores refletivos (emissor IR + fototransistor) com **saída analógica**, um por canal do ADC. O modelo da barra **não é identificado no repositório**. Pelo padrão do firmware (`SENSOR_INVERT = 0`), a linha produz tensão **alta** no ADC. Se a barra usada der tensão baixa sobre a linha, basta mudar `SENSOR_INVERT` para 1.

- Cada sensor tem peso 0, 1000, ..., 7000; a posição é a média ponderada.
- A ordem física pode ser invertida por `SENSOR_ORDEM_INVERTIDA` (atualmente **1**: S0/PA0 fica do lado esquerdo).
- Os valores padrão de calibração (`SENSOR_MIN_DEFAULT = 400`, `SENSOR_MAX_DEFAULT = 3600`) são provisórios; o código diz isso explicitamente.

3.3 Driver de motores – DRV8833

Ponte H dupla da Texas Instruments (2,7–10,8 V, até ~1,5 A RMS por canal, segundo o datasheet). Cada motor usa dois pinos de entrada: um ligado a um canal PWM e outro a um GPIO. A tabela-verdade usada pelo firmware (`motor.c`) é:

PWM (INx)	GPIO (INy)	Estado do motor	Uso no firmware
0	0	Solto (coast)	Calibração (<code>Motor_Solto</code>)
PWM	0	Sentido A, fast decay	Comando positivo se <code>INVERTIDO = 0</code>
PWM invertido	1	Sentido B, slow decay	Comando positivo se <code>INVERTIDO = 1</code>
1	1	Freio	Comando 0, STOP

Com `MOTOR_ESQ_INVERTIDO = MOTOR_DIR_INVERTIDO = 1` (configuração atual), os dois motores andam para frente em **slow decay**: no tempo desligado do PWM o motor fica em freio em vez de solto. Isso deixa a velocidade mais proporcional ao duty e melhora a resposta a baixa velocidade.

Deteção do pino de direção. Na inicialização o firmware confere no registrador `MODER` se PB4 e PB6 estão como saída. Se algum não estiver (por exemplo, porque o CubeMX foi alterado), aquele motor perde ré e freio (0 vira “solto”) e o STM32 envia `ERR MOTOR ESQ/DIR ...` pela serial.

3.4 Motores

Dois motores DC com redução, um por roda. Modelo, tensão nominal e relação de redução **não constam no repositório**. A zona morta medida provisoriamente é de 300 U (30% de duty) para cada roda (DZ_ESQ, DZ_DIR); o firmware tem um modo próprio para medi-la (MOD0_MEDIR_DZ, seção 4.6).

3.5 ESP32-C3

Placa ESP32-C3 (ex.: “Super Mini”) programada pelo Arduino IDE. Usa a UART1 nos pinos GPIO6 (RX) e GPIO7 (TX), cria um ponto de acesso Wi-Fi próprio e serve uma página em <http://192.168.4.1>. Também repassa o Serial USB para o STM32, então dá para mandar comandos pelo Serial Monitor. Os dois lados usam lógica de 3,3 V, sem conversor de nível. Evite os pinos GPIO 2, 8 e 9 (pinos de *strapping*).

3.6 Alimentação

Carregador. A bateria do robô é carregada por um módulo carregador **LX LISC V1**.

Capacitores de desacoplamento. O robô usa capacitores de desacoplamento entre os seus terminais. Os motores DC e o chaveamento do PWM a 20 kHz geram picos e quedas de tensão; os capacitores absorvem esses transientes perto de onde eles surgem e evitam que o ruído chegue ao STM32, à barra de sensores e ao ESP32-C3. Isso reduz o risco de reset do microcontrolador e de leituras erradas no ADC quando os motores aceleram ou freiam.

Outros cuidados obrigatórios:

- O GND do STM32, do DRV8833, da barra de sensores e do ESP32-C3 precisa ser comum.
- A tensão dos motores precisa estar dentro da faixa do DRV8833 (2,7–10,8 V).

3.7 Pinagem completa

Pino	Função	Periférico / config	Liga em
PA0..PA7	Sensores S0..S7	ADC1_IN0..IN7, ranks 1..8	Barra, canais 0..7
PB5	PWM motor direito	TIM3_CH2, AF2	DRV8833 IN1
PB4	Direção motor direito	GPIO saída (MOTOR2_DIR)	DRV8833 IN2
PB3	PWM motor esquerdo	TIM2_CH2, AF1	DRV8833 IN3
PB6	Direção motor esquerdo	GPIO saída (MOTOR_DIR_R)	DRV8833 IN4
PB2	<i>não usado</i>	GPIO saída (MOTOR1_DIR) no CubeMX	–
PA9	UART TX → ESP32	USART1_TX, AF7	ESP32-C3 GPIO6
PA10	UART RX ← ESP32	USART1_RX, AF7, pull-up	ESP32-C3 GPIO7
PA13 / PA14	SWDIO / SWCLK	Depuração	ST-Link
GND	Terra comum	–	ESP32, DRV8833, sensores

4. Firmware do STM32

4.1 Organização dos arquivos

O projeto segue a estrutura do STM32CubeIDE. Os arquivos gerados pelo CubeMX (adc.c, tim.c, gpio.c, dma.c, usart.c, stm32f4xx_it.c) só configuram periféricos. A lógica do robô está em quatro arquivos:

Arquivo	Responsabilidade
main.c	Inicialização, laço principal, máquina de estados, partida do ADC/DMA, recuperação de overrun, modo de medição de zona morta.
control.c / control.h	Calibração e normalização dos sensores, cálculo de posição, memória de linha perdida, PID, mistura das rodas e rampa.

Arquivo	Responsabilidade
motor.c / motor.h	Camada do DRV8833: compensação de zona morta, conversão U → CCR, sentido, freio e roda solta.
comm.c / comm.h	UART por interrupção com buffers circulares, interpretador de comandos ASCII, watchdog do link.
robot_config.h	Todos os parâmetros ajustáveis de motores, mistura, rampa e linha perdida.

4.2 Temporização e laço principal

O TIM4 gera uma interrupção a cada 2 ms que só liga a flag `tick_flag`. Todo o trabalho é feito no laço principal, fora de interrupção:

```
while (1) {
    Comm_Process();           // executa comandos que chegaram pela UART
    Sensors_CheckADC();       // reinicia ADC/DMA se houver overrun
    if (tick_flag) {          // a cada 2 ms
        tick_flag = false;
        // watchdog do link -> STOP
        // switch (robot_state) { ... seguir linha / parado / teste / calibração }
    }
}
```

Prioridades de interrupção: TIM4 = 0 (mais alta), USART1 = 1. As interrupções de meia e fim de transferência do DMA são desligadas de propósito: com o ADC em modo contínuo elas disparariam a cada poucos microssegundos e travariam o laço e a UART.

4.3 Aquisição dos sensores

O ADC1 converte os 8 canais em sequência, sem parar, e o DMA2 grava cada resultado em `line_sensor_raw[]`. Com ADC a 21 MHz e 156 ciclos por canal (144 de amostragem + 12 de conversão), cada varredura leva $\approx 59 \mu\text{s}$, ou seja, cerca de 34 varreduras novas a cada tick de controle. O laço sempre lê o valor mais recente, sem esperar.

Se ocorrer *overrun* (o DMA não leu um resultado a tempo), o ADC para e os valores congelariam. `Sensors_CheckADC()` detecta a flag OVR, reinicia ADC + DMA e envia EVT_ADC_RESTART.

4.4 Normalização, posição e linha perdida

Normalização

Cada leitura bruta é mapeada para 0..1000 com o mínimo e máximo calibrados daquele sensor:

```
n = clamp( (raw - min[i]) * 1000 / (max[i] - min[i]), 0, 1000 )
se SENSOR_INVERT: n = 1000 - n
se n < SENSOR_NOISE_FLOOR (50): n = 0 // ignora o "quase branco"
```

Posição

A posição é a média ponderada dos sensores normalizados com pesos 0, 1000, ..., 7000. O piso de ruído é importante: sem ele, sensores fora da linha com valores pequenos puxariam a média para o centro e reduziriam a correção.

```
posição =  $\sum (n[i] \cdot \text{peso}[i]) / \sum n[i]$  // 0..7000
linha vista = algum n[i] > LIMAR_LINHA (200)
erro = 3500 - posição
```

Linha perdida

Quando nenhum sensor passa do limiar, `Linha_Erro()` não para o robô; ele usa a memória do último erro válido:

- Se $|\text{último erro}| > \text{MARGEM_LADO}$ (1050, 30% do máximo): a linha saiu por um lado, então usa $\text{erro} = \pm 3500$ para girar forte para aquele lado.
- Senão: o robô estava quase reto (falha na fita, cruzamento); mantém o último erro e segue.

- Ao reencontrar a linha, `prev_error` do PID recebe o erro atual, para não gerar um pico no termo derivativo.
- Após 10 ticks seguidos (20 ms) sem linha, o estado passa a `LOST` só para informação; o comportamento é o mesmo.

4.5 Controlador PID

$P = K_p \cdot e$
 $I = K_i \cdot \Sigma e$ (Σe limitado para $|K_i \cdot \Sigma e| \leq \text{integral_max} = 1000$)
 $D = K_d \cdot (e - e_{\text{anterior}})$
 $\text{saída} = P + I + D$

- Executado a cada 2 ms em `float`. O termo D usa a diferença por tick, sem dividir pelo tempo; por isso o valor de `Kd` depende da frequência do laço.
- Anti-windup: o acumulador é limitado de forma que a contribuição do termo I nunca passe de 1000. Com `Ki = 0` o acumulador é zerado.
- Os ganhos estão na escala antiga do PWM (duty 0..4199). A saída é convertida para a escala U por $\text{PID_PARA_U} = 1000 / 4199 \approx 0,238$, assim os ganhos antigos mantêm o efeito.
- `PID_Reset()` zera integral e erro anterior; é chamado no `RUN` e sempre que um ganho muda.

4.6 Dos números ao motor: mistura, rampa e zona morta

Cada tick de 2 ms (TIM4):

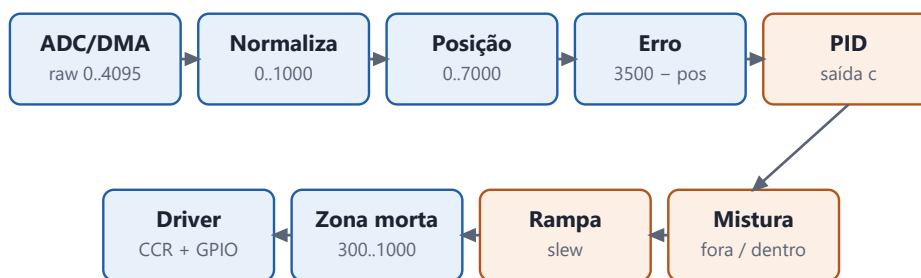


Figura 2 – Caminho do sinal em cada tick de controle. Em laranja, as etapas de controle e suavização do comando.

Mistura das rodas (`Motors_ApplyPID`)

```

c      = clamp(saída_PID * 0,238, -2000, 2000)
vb     = vel_base   (opcional: reduzida por K_REDUCAO_CURVA * |erro|)
fora   = vb
dentro = max(vb - 2*|c|, 0)           // nunca negativa: sem ré
c ≥ 0 → direita = fora, esquerda = dentro
c < 0 → esquerda = fora, direita = dentro

```

A roda de fora nunca passa da velocidade base, então a curva nunca é mais rápida que a reta. Toda a correção é feita freando a roda de dentro, até pará-la; a roda nunca recebe comando negativo (ré).

Rampa (limite de variação)

Cada roda tem sua própria rampa. O comando pode subir no máximo `SLEW_SUBIDA = 40 U` por tick (0 → 100% em 50 ms) e descer no máximo `SLEW_DESCIDA = 60 U` por tick (100% → 0 em ≈ 34 ms). Isso filtra o ruído do PID, que fazia as rodas tremerem. A descida é mais rápida para a curva não atrasar. Ao entrar em seguir linha, `Motors_ResetRampa()` faz as duas rodas partirem de 0.

Compensação de zona morta (`motor.c`)

Abaixo de ~30% de duty os motores não vencem a inércia. Por isso o comando é remapeado:

```

|u| < U_MIN_ATIVO (20) → 0 (freio)
|u| em 1..1000 → DZ + (|u| - 1) · (1000 - DZ) / 999 // DZ = 300
exemplo: 20 → 313 500 → 650 1000 → 1000

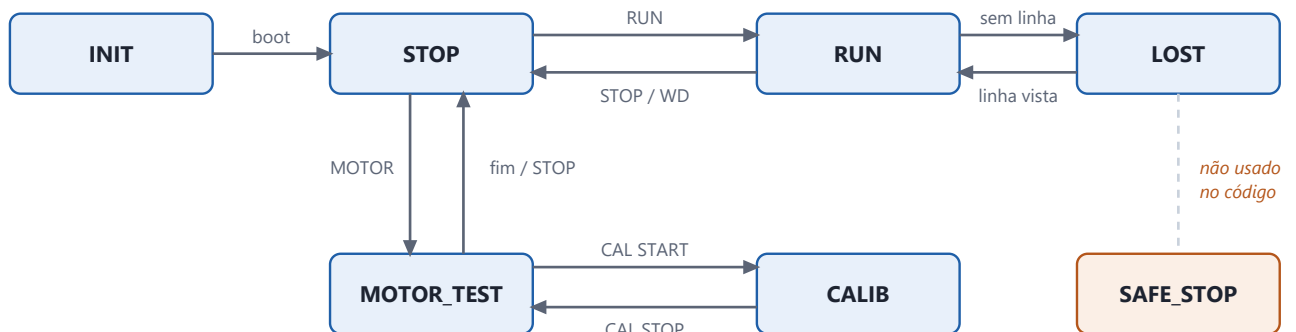
```

Saída no hardware

O valor final é convertido para o registrador de comparação do timer lendo o ARR em tempo de execução ($CCR = u \cdot (ARR + 1) / 1000$). Conforme o sinal e o INVERTIDO de cada motor, o GPIO fica em LOW ou HIGH e o CCR é usado direto ou invertido (tabela 3.3).

Modo de medição de zona morta. Com `MODO_MEDIR_DZ = 1`, o comando RUN, em vez de seguir linha, sobe os dois motores de 0 a 1000 em passos de 10 U a cada 300 ms, sem compensação, e envia DZ <u> a cada passo. O primeiro valor em que a roda gira com o robô no chão é o DZ_ESQ/DZ_DIR correto.

4.7 Máquina de estados



RUN e LOST são o mesmo comportamento (seguir linha); LOST só muda o nome no STATUS.
STOP pode ser enviado de qualquer estado; o watchdog do link (WD) também leva a STOP.

Figura 3 – Estados do robô (`RobotState_t`) e transições.

Estado (STATUS)	Motores	Entra por	Sai por
INIT	Freio	Boot	Vai para STOP (ou RUN após <code>AUTO_START_DELAY_MS</code> , hoje 0 = desligado)
STOP	Freio	Comando STOP, watchdog, fim de teste/calibração	RUN, MOTOR, CAL START
RUN / LOST	PID	Comando RUN	STOP, watchdog
MOTOR_TEST	Valor fixo do comando	MOTOR l r ms	Tempo acabou (EVT MOTOR DONE), STOP
CALIB	Soltos (coast)	CAL START	CAL STOP, STOP
SAFE_STOP	Freio	<i>nenhum caminho no código atual</i>	–

4.8 Protocolo serial

ASCII, uma linha por comando terminada em `\n` (o `\r` é ignorado), separadores espaço ou vírgula, sem diferenciar maiúsculas. A recepção é por interrupção em um buffer circular de 128 bytes; a transmissão usa um buffer de 512 bytes e nunca bloqueia: se encher, a resposta é descartada para não atrasar o controle.

Comando	Resposta	Efeito
PING	PONG	Testa o link
STATUS	STATUS <estado> KP ... KI ... KD ... SPEED <u>	Estado e parâmetros
STOP	OK STOP	Freia na hora
RUN	OK RUN	Zera PID e memória, começa a seguir linha

Comando	Resposta	Efeito
PID kp ki kd	OK PID ...	Muda os 3 ganhos (–10000..10000, rejeita nan/inf)
KP / KI / KD v	OK PID ...	Muda um ganho
PID?	PID kp ki kd	Lê os ganhos
SPEED u	OK SPEED u	Velocidade base 0..1000
MOTOR l r ms	OK MOTOR ... e depois EVT MOTOR DONE	Teste com –1000..1000 por 1..10000 ms (aceita ré)
SENS	SENS R ... N ... POS p LINE 0 1	Brutos, normalizados, posição usada pelo PID
CAL START / STOP	OK CAL START / CAL MIN ... MAX ... [REJ ...]	Calibração (seção 4.9)
CAL?	CAL MIN ... MAX ... [WARN]	Lê a calibração
WD ms	OK WD ms	Watchdog do link, 0..60000 ms (0 = desligado)
HB	<i>sem resposta</i>	Só alimenta o watchdog

Eventos espontâneos: HELLO LINEFOLLOWER (boot), EVT MOTOR DONE, EVT WD STOP, EVT ADC RESTART. O printf do newlib-nano não formata float; por isso os ganhos são impressos por uma função própria com 5 casas decimais.

4.9 Calibração dos sensores

- CAL START: solta os motores, guarda a calibração atual como backup e zera min/max.
- A cada tick, LineSensor_CalibSample() atualiza o mínimo e o máximo de cada sensor enquanto o robô é arrastado à mão sobre a linha.
- CAL STOP: sensores com faixa (max – min) menor que CAL_MIN_RANGE (200) são rejeitados e voltam ao backup; os índices aparecem após REJ.

A calibração e os ganhos ficam só na RAM e se perdem ao desligar. Quando os valores estiverem bons, copie-os para Control_Init(...) em main.c e para SENSOR_MIN_DEFAULT / SENSOR_MAX_DEFAULT.

5. Firmware do ESP32-C3 (ESP32C3_Tuner)

Sketch Arduino sem bibliotecas externas (WiFi, WebServer, Preferences vêm com o core ESP32). Ele cria a rede LineFollower em modo AP e serve uma página única com estas seções: STOP/RUN, PID (enviar, ler do robô, salvar no ESP), velocidade base em %, teste de motores, sensores (leitura, modo ao vivo, calibração), comando manual e log das últimas 40 linhas.

Rota HTTP	Função
/	Página HTML (embutida em PROGMEM)
/cmd?c=...	Envia um comando ao STM32 e devolve a resposta (timeout 200 ms; ignora EVT/HELLO)
/save	Grava Kp, Ki, Kd e velocidade na Flash do ESP (Preferences)
/hb?on=1	Heartbeat da página (a cada 300 ms)
/log	Log em JSON para a página

Restauração automática

Quando o STM32 reinicia ele envia HELLO LINEFOLLOWER; o ESP então reenvia o PID e a velocidade salvos (a calibração dos sensores **não** é restaurada). Valores salvos na escala antiga (> 1000) são convertidos.

Segurança em duas camadas

- **Página → ESP:** com a opção “Parar o robô se esta página desconectar” marcada, se o heartbeat da página parar por 2,5 s o ESP envia STOP.
- **ESP → STM32:** na mesma situação o ESP liga WD 1000 no STM32 e envia HB a cada 300 ms. Se o ESP travar ou o fio soltar, o próprio STM32 para em 1 s e envia EVT WD STOP.

- Com a opção desmarcada (ou sem o ESP), não há watchdog: o robô segue até receber STOP.

6. Parâmetros ajustáveis

Parâmetros de compilação (mudam com regravação). Os marcados com * também podem ser mudados em tempo real pela serial.

Parâmetro	Valor	Arquivo	Significado
Kp, Ki, Kd *	0,1 · 0 · 0,05	main.c	Ganhos iniciais do PID
VEL_BASE_PADRAO *	595 U	robot_config.h	Velocidade base (SPEED)
DZ_ESQ / DZ_DIR	300 U	robot_config.h	Zona morta de cada motor
U_MIN_ATIVO	20 U	robot_config.h	Abaixo disso o comando vira freio
SLEW_SUBIDA	40 U/tick	robot_config.h	Aceleração máxima da rampa
SLEW_DESCIDA	60 U/tick	robot_config.h	Desaceleração máxima da rampa
K_REDUCAO_CURVA	0 (desligado)	robot_config.h	Reduz vel. base com o erro
VEL_MIN_CURVA	300 U	robot_config.h	Piso da redução acima
MOTOR_*_INVERTIDO	1 / 1	robot_config.h	Sentido e tipo de decay
LIMIAR_LINHA	200	robot_config.h	Normalizado acima disso = vê linha
MARGEM_LADO	1050	robot_config.h	Decide giro forte ao perder a linha
SENSOR_INVERT	0	control.h	1 se a linha der valor baixo
SENSOR_ORDEM_INVERTIDA	1	control.h	Ordem física da barra
SENSOR_NOISE_FLOOR	50	control.h	Piso de ruído na média
CAL_MIN_RANGE	200	control.h	Faixa mínima aceita na calibração
AUTO_START_DELAY_MS	0	main.c	Partida automática (0 = espera RUN)
LOST_DEBOUNCE_TICKS	10	main.c	Ticks sem linha até mostrar LOST
MODO_MEDIR_DZ	0	robot_config.h	1 = RUN mede zona morta

7. Procedimentos de uso

7.1 Primeira ligação

- Grave o STM32 pelo STM32CubeIDE (ST-Link em PA13/PA14) e o ESP32-C3 pelo Arduino IDE.
- Conecte-se à rede LineFollower e abra <http://192.168.4.1>.
- Com as rodas fora do chão, use o teste de motores com valores positivos. As duas rodas precisam girar para frente; se uma girar ao contrário, troque o MOTOR_*_INVERTIDO dela.
- Confira o lado de cada motor com “Só esquerdo” e “Só direito”.

7.2 Sensores

- Clique em “Ao vivo”. Sobre a linha as barras devem subir; se descerem, mude SENSOR_INVERT.
- Com a linha sob o lado **direito** da barra, SENS deve dar POS perto de 0. Perto de 7000: troque SENSOR_ORDEM_INVERTIDA.
- Calibre: “Calibrar: iniciar”, arraste o robô devagar de um lado ao outro da linha, “Calibrar: terminar”. Repita se aparecer REJ.

7.3 Sintonia do PID

- Comece com SPEED 450 e KI 0.

- Suba K_P (0,4 → 0,6 → 0,8) até o robô voltar rápido para a linha; pare quando começar a zigue-zaguear na reta. Com $K_P \approx 0,7$, a roda de dentro para quando o erro chega à metade da barra.
- Suba K_D (2 → 5 → 10) para amortecer o zigue-zague. Se as rodas zumbirem ou tremerem, o K_d passou do ponto.
- Aumente `SPEED` aos poucos e reajuste. Salve no `ESP` e, no final, copie para `main.c`.

8. Referências

- STMicroelectronics – STM32F401xB/C datasheet e RM0368 (manual de referência).
- Texas Instruments – DRV8833 Dual H-Bridge Motor Driver, datasheet (SLVSAR1).
- Espressif – ESP32-C3 Series Datasheet; Arduino-ESP32 core (WiFi, WebServer, Preferences).
- Código-fonte: `LineFollowerRobot/Core/Src`, `LineFollowerRobot/Core/Inc`, `ESP32C3_Tuner/ESP32C3_Tuner.ino`, `PINOUT.md`.